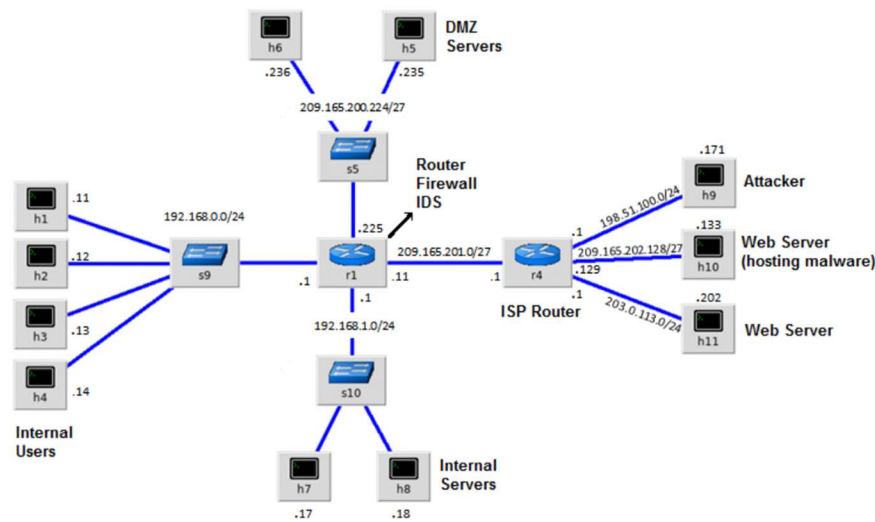


Snort and Firewall Rules

[Detecting malicious activities using Snort IDS → remediating with Firewall policies]



[Network Topology]

Snort IDS

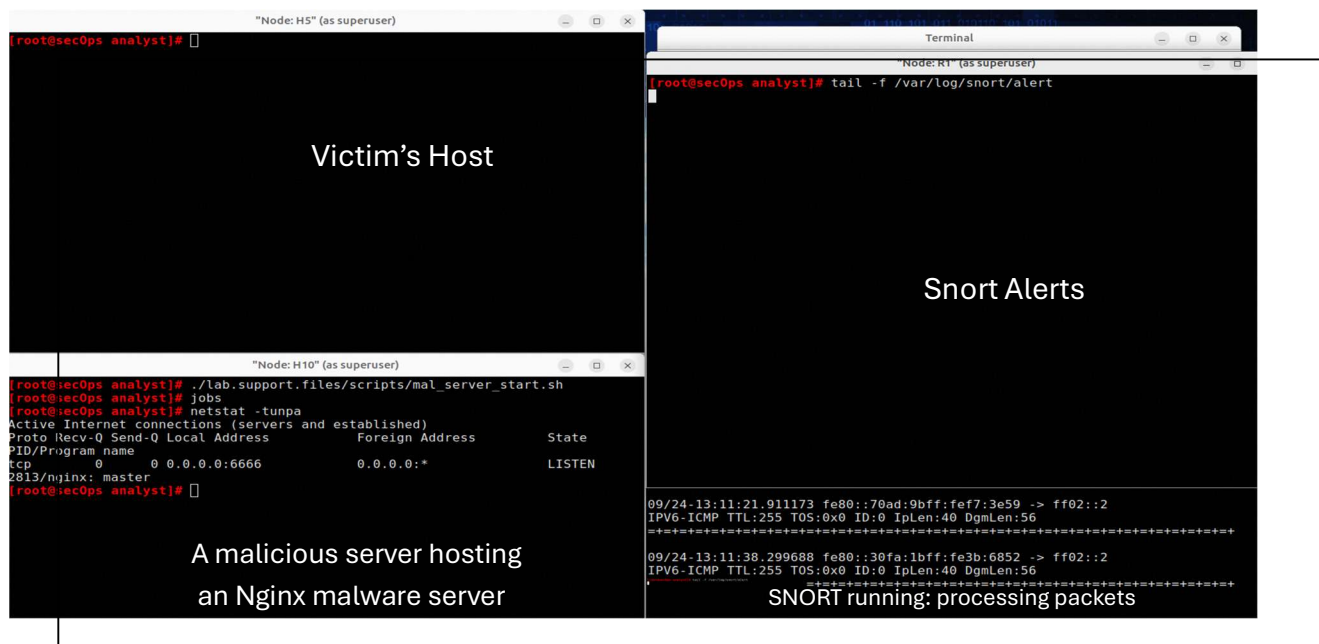
After initializing the network, and accessing R1 Router as *root* (as shown from the shell prompt):

```
[analyst@secOps ~]$ sudo ./lab.support.files/scripts/cyberops_extended_topo_no_f
w.py
*** Adding controller
*** Add switches
*** Add hosts
*** Add links
*** Starting network
*** Configuring hosts
R1 R4 H1 H2 H3 H4 H5 H6 H7 H8 H9 H10 H11
*** Starting controllers
*** Starting switches
*** Add routes
*** Post configure switches and hosts
*** Starting CLI:
mininet> xterm R1
mininet> [ ]
```

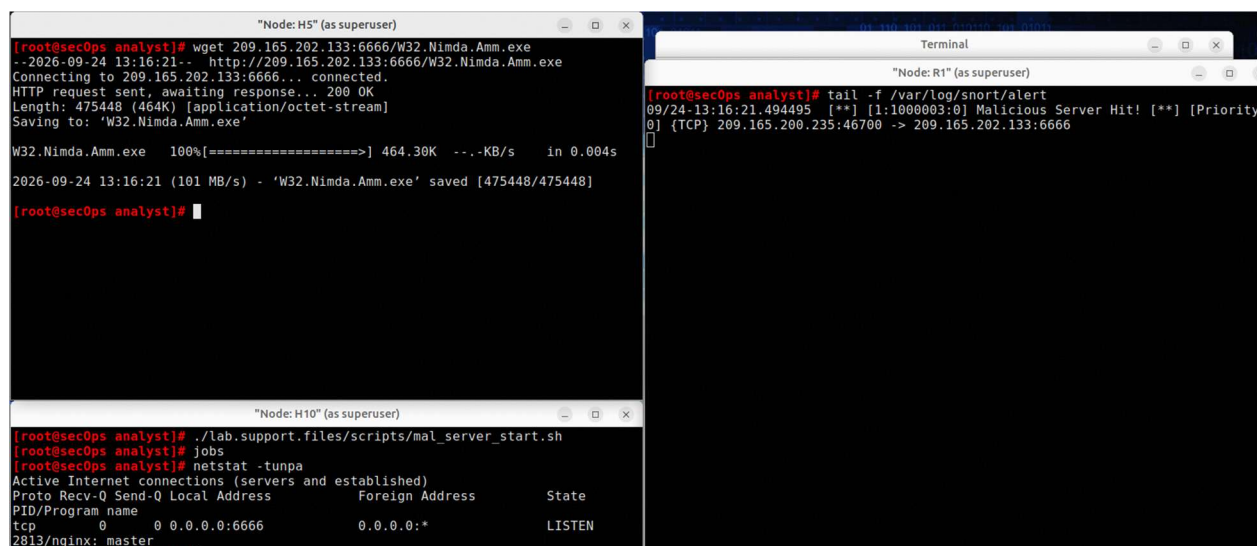
```
[root@secOps analyst]#
[root@secOps analyst]#
```

I started the IDS snort: **Commencing packet processing (pid=2777)**

Now we have:



Let's try to download malware as the victim from the malicious server, knowing that packets have to go through the router running Snort according to our topology:

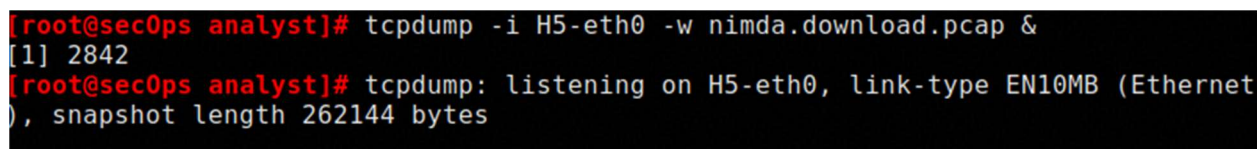


And the **Malicious server** is detected!!!

Communicating from 46700 port to remote 6666 port

Unfortunately the file was completely downloaded

Let's use the tcpdump command to capture the event and download the malware file again to capture the transaction:



And retried downloading the file from the malicious server, which again triggered the same alert

```
[root@secOps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
--2026-09-24 13:27:43-- http://209.165.202.133:6666/W32.Nimda.Amm.exe
Connecting to 209.165.202.133:6666... connected.
HTTP request sent, awaiting response... 200 OK
Length: 475448 (464K) [application/octet-stream]
Saving to: 'W32.Nimda.Amm.exe.1'

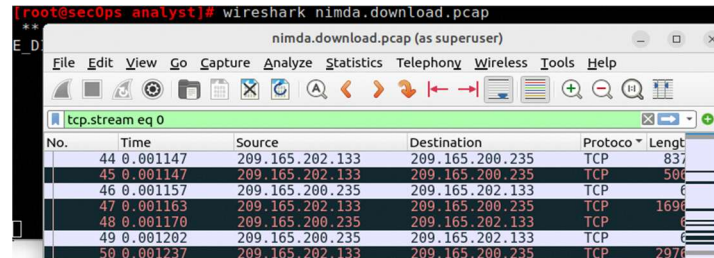
W32.Nimda.Amm.exe.1 100%[=====] 464.30K --.-KB/s in 0.001s

2026-09-24 13:27:43 (541 MB/s) - 'W32.Nimda.Amm.exe.1' saved [475448/475448]

[root@secOps analyst]# fg tcpdump
tcpdump -i H5-eth0 -w nimda.download.pcap
^C61 packets captured
61 packets received by filter
0 packets dropped by kernel

[root@secOps analyst]# tail -f /var/log/snort/alert
09/24-13:16:21.494495 [**] [1:1000003:0] Malicious Server Hit! [**] [Priority 0] (TCP) 209.165.200.235:46700 -> 209.165.202.133:6666
09/24-13:27:43.836912 [**] [1:1000003:0] Malicious Server Hit! [**] [Priority 0] (TCP) 209.165.200.235:58416 -> 209.165.202.133:6666
```

But let's see what's the trigger of the alert found in the captured packets in the pcap file, with wireshark:



Firewall rules based on Snort IDS detection

Now that the server is deemed Malicious, let's block access to it using **IPTABLES** firewall policies running on the R1 router as well:

```
[root@secOps analyst]# iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

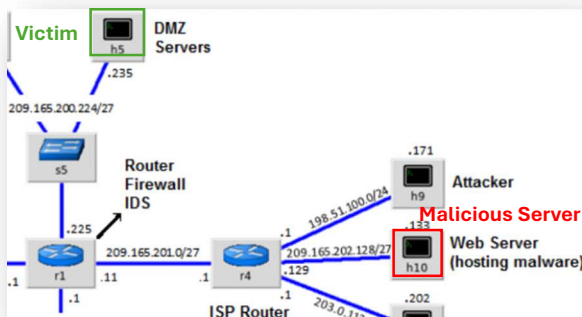
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination
```

Iptables uses the concepts of chains (INPUT, OUTPUT, FORWARD) and rules to filter traffic.

In our case above, the three chains have no explicit rules, however each has an implicit 'ACCEPT ALL' rule.

So let's make a rule blocking all access to and from the malicious server



The rules:

```
[root@sec0ps analyst]# iptables -I FORWARD -p tcp -d 209.165.202.133 --dport 6666 -j DROP
[root@sec0ps analyst]# iptables -I FORWARD -p tcp -s 209.165.202.133 -j DROP
```

The firewall policy table becomes:

```
[root@sec0ps analyst]# iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
    0    0 DROP      tcp  --  any    any    209.165.202.133         anywhere
    0    0 DROP      tcp  --  any    any    anywhere                209.165.202.133      tcp dpt:6666

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
```

This should block our previous test from happening, let's try:

```
[root@sec0ps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
--2026-09-24 13:57:36-- http://209.165.202.133:6666/W32.Nimda.Amm.exe
Connecting to 209.165.202.133:6666... failed: Connection timed out.

[root@sec0ps analyst]# tail -f /var/log/snort/alert
```

And our rule successfully stopped H5 from connecting to the malicious server as shown above.

⇒ A more **aggressive yet valid** ruleset would be blocking every possible packet from and to the malicious server (any port, any protocol) and on all chains (INPUT, OUTPUT, FORWARD).